ALGORITMIA

Grado en Ingeniería Informática del Software

PRÁCTICA 6 Opcional

OBJETIVO:

- **Backtracking**

El laberinto

Se dispone de un laberinto en el que hay un animal (por ejemplo un ratón). La idea de la práctica es conseguir que el animal salga del laberinto. Es decir, comenzará en una posición inicial dada y tendrá que salir a través de una posición final indicada.

El laberinto puede ser visto como una tabla en la que el camino disponible se puede representar con un número (por ejemplo el 0), el camino seguido por el aminal puede representarse con otro número (por ejemplo el 2), y los obstáculos o paredes que el animal no puede atravesar podría representarte con otro número (por ejemplo el 1).

TRABAJO PEDIDO

Describir e implementar en Java el algoritmo que solucione el problema utilizando Backtracking:

- LaberintoTodas.java → Debe obtener todas las soluciones posibles, mostrarlas según se vayan produciendo y finalmente mostrar la mejor solución (o una de las mejores).

Los datos se cargarán desde un fichero de texto con laberintos de tamaño 7x7. Un ejemplo de ejecución podría ser:

LaberintoTodas caso1 0 48

Lo que significaría que queremos la primera solución utilizando como ejemplo el laberinto guardado en el fichero **caso1.txt** Además, indicamos que el animal comienza en la primera posición del laberinto (arriba en la izquierda) y debe salir por la última posición del laberinto (abajo a la derecha).

El resultado de la ejecución del programa podría ser como sigue:

```
EL LABERINTO ES INICIALMENTE DEL SIGUIENTE MODO:
  0  0  0  0  0  0  0
  1  1  1  1  1  1  0
  0  0  0  0  0  0  0
  0  1  1  1  1  1  1
  0  1  1  1  1  1  0
  0  0  0  0  0  1  0
  0  0  1  0  0  0  0

El objetivo es ir desde la posición 0 a la posición 48
```

El resultado obtenido podría ser el siguiente:

```
EL LABERINTO ES INICIALMENTE DEL SIGUIENTE MODO:
  0  0  0  0  0  0  0
  1  1  1  1  1  1  0
  0  0  0  0  0  0  0
  0  1  1  1  1  1  1
  0  1  1  1  1  1  0
  0  0  0  0  0  1  0
  0  0  1  0  0  0  0

El objetivo es ir desde la posición 0 a la posición 48

SOLUCIÓN ENCONTRADA CON 26 PASOS
  2  2  2  2  2  2  2
  1  1  1  1  1  1  2
  2  2  2  2  2  2  2
  2  1  1  1  1  1  1
  2  1  1  1  1  1  0
  2  2  2  2  0  1  0
  2  2  1  2  2  2  2

SOLUCIÓN ENCONTRADA CON 26 PASOS
  2  2  2  2  2  2  2
  1  1  1  1  1  1  2
  2  2  2  2  2  2  2
  2  1  1  1  1  1  1
  2  1  1  1  1  1  0
  2  2  2  2  2  1  0
  2  2  1  0  2  2  2

SOLUCIÓN ENCONTRADA CON 24 PASOS
  2  2  2  2  2  2  2
  1  1  1  1  1  1  2
  2  2  2  2  2  2  2
  2  1  1  1  1  1  1
  2  1  1  1  1  1  0
  2  2  2  2  0  1  0
  0  0  1  2  2  2  2

SOLUCIÓN ENCONTRADA CON 24 PASOS
  2  2  2  2  2  2  2
  1  1  1  1  1  1  2
  2  2  2  2  2  2  2
  2  1  1  1  1  1  1
  2  1  1  1  1  1  0
  2  2  2  2  2  1  0
  0  0  1  0  2  2  2

LA MEJOR SOLUCIÓN TIENE 24 PASOS, HABIENDO UN TOTAL DE 4 SOLUCIONES
DIFERENTES
  2  2  2  2  2  2  2
  1  1  1  1  1  1  2
  2  2  2  2  2  2  2
  2  1  1  1  1  1  1
  2  1  1  1  1  1  0
  2  2  2  2  0  1  0
  0  0  1  2  2  2  2
```

Para hacer la visualización se propone que a la hora de mostrar el array con el laberinto, se hagan las siguientes sustituciones: 0 por "·", 1 por "H", 2 por "*" y por tanto la última solución se visualizaría:

```
*  *  *  *  *  *  *
H  H  H  H  H  H  *
*  *  *  *  *  *  *
*  H  H  H  H  H  H
*  H  H  H  H  H  ·
*  *  *  *  ·  H  ·
·  ·  H  *  *  *  *
```